

CO1 - AT1

Course Code: MLA0201

Name: Siva Balan K

Course Name: Fundamentals of Machine Learning

Reg.no: 192572037

Application Based Questions

Q1) Implement the concept of probability distribution to predict the likelihood of spam emails in mail filtering system.

Problem Description

The objective is to build a mail-filtering system that predicts whether an incoming email is spam or legitimate. Email text is converted into numerical features using TF-IDF. Multinomial Naive Bayes applies prior and conditional probabilities to calculate the likelihood of the email belonging to the spam or non-spam class.

The model predicts:

- 0 – Not spam or legitimate email
- 1 – Spam email

Dataset Description

The **Spam Email Classification Dataset** was uploaded by **Puru Singhvi**. It combines email messages from the TREC 2007 Public Spam Corpus and the Enron-Spam dataset. It contains the columns text and label. The Kaggle description lists 83,446 labelled emails, while the downloaded CSV used during this experiment loaded with 83,448 rows.

Dataset source: [Spam Email Classification Dataset – Kaggle](#)

Algorithm / Pseudocode

1. Load the spam-email dataset from the CSV file.
2. Remove missing and duplicate email records.
3. Divide the dataset into training and testing sets.
4. Convert email text into TF-IDF numerical features.
5. Train a Multinomial Naive Bayes classifier.

6. Evaluate the model using accuracy and a confusion matrix.
7. Predict spam probability for a new email entered by the user.

Python Code

```
from pathlib import Path

import pandas as pd from sklearn.feature_extraction.text
import TfidfVectorizer from sklearn.metrics import (
accuracy_score, classification_report,
confusion_matrix
)
from sklearn.model_selection import train_test_split from
sklearn.naive_bayes import MultinomialNB

print("Spam Email Detection Program Started")

# Locate dataset folder =
Path(__file__).resolve().parent dataset_path = folder /
"spam-email-classification.csv"

if not dataset_path.exists():
    print("Dataset not found:", dataset_path)
raise SystemExit

# Load dataset data
= pd.read_csv(
dataset_path,
encoding_errors="i
```

```

gnore",
on_bad_lines="skip"
p"
)

print("Dataset loaded successfully") print("Original
dataset size:", data.shape)

# Select and clean columns data =
data[["text", "label"]].copy() data =
data.dropna() data =
data.drop_duplicates(subset=["text"])

data["text"] = data["text"].astype(str)
data["label"] = pd.to_numeric(
data["label"], errors="coerce"
)

data = data.dropna() data["label"] =
data["label"].astype(int) data =
data[data["label"].isin([0, 1])]

print("Dataset size after cleaning:", data.shape) print("\nClass
distribution:") print(data["label"].value_counts())

# Input and target
X = data["text"] y
= data["label"] #
Split dataset

```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42, stratify=y
)

# Convert text into TF-IDF features vectorizer
= TfidfVectorizer( stop_words="english",
    max_features=10000
)

X_train_vector = vectorizer.fit_transform(X_train)
X_test_vector = vectorizer.transform(X_test)

# Train Naive Bayes model model
= MultinomialNB()
model.fit(X_train_vector, y_train)

print("\nModel trained successfully")

# Evaluate model predictions =
model.predict(X_test_vector)

print(
    "Model accuracy:", round(accuracy_score(y_test,
    predictions) * 100, 2),
    "%"
)
```

```
print("\nConfusion Matrix:") print(confusion_matrix(y_test,
predictions))
```

```
print("\nClassification Report:") print(
classification_report(    y_test,
predictions,    target_names=["Not
Spam", "Spam"],    zero_division=0
    )
)
```

```
# Predict a new email new_email =
input("\nEnter an email message: ")
```

```
new_email_vector = vectorizer.transform([new_email])
```

```
prediction = model.predict(new_email_vector)[0] probability
= model.predict_proba(new_email_vector)[0]
```

```
not_spam_probability = probability[0] * 100 spam_probability
= probability[1] * 100 print(
```

```
    "\nNot-Spam Probability:",
round(not_spam_probability, 2),
    "%")
)
```

```
print(    "Spam Probability:",
round(spam_probability, 2),
    "%")
)
```

```

if prediction == 1:

    print("Final Result: SPAM EMAIL")

else:

    print("Final Result: NOT SPAM EMAIL")

```

Output Screenshot

```

Spam Email Detection Program Started
Dataset found: spam-email-classification.csv
Dataset loaded successfully
Original dataset size: (83448, 2)
Dataset size after cleaning: (83446, 2)

Class distribution:
label
1    43908
0    39538
Name: count, dtype: int64
0 = Not Spam
1 = Spam

Training records: 66756
Testing records: 16690

Converting email text into numerical features...
Text conversion completed
Training the Naive Bayes model...
Model trained successfully

Model Accuracy: 97.12 %

Confusion Matrix:
[[7737  171]
 [ 310 8472]]

Classification Report:
              precision    recall  f1-score   support

   Not Spam       0.96       0.98       0.97       7908
     Spam        0.98       0.96       0.97       8782

   accuracy              0.97       0.97       0.97      16690
  macro avg              0.97       0.97       0.97      16690
weighted avg              0.97       0.97       0.97      16690


----- Test a New Email -----
Enter an email message: hi am your friend siva send me 10000 to this gpay number 1234567xxx

Not-spam probability: 40.99 %
Spam probability: 59.01 %
Final Result: SPAM EMAIL

```

Result

The spam-mail filtering system was implemented successfully using TF-IDF and Multinomial Naive Bayes. The model calculated the conditional probability of an email belonging to the spam and non-spam classes and classified the email according to the class with the greater posterior probability.

Q3) Develop the designing steps of a learning system to build a ML solution for students performance prediction.

Problem Description

The objective is to design a machine-learning system that predicts whether a student is likely to pass or be academically at risk. The system learns from previous student records containing attendance, study hours, previous scores, sleep hours, tutoring sessions and physical activity.

A performance label is created as:

- 0 – At Risk
- 1 – Pass

Students with an Exam_Score of 60 or above are labelled as passing.

Dataset Description

The **Student Performance Factors** dataset contains information about study habits, attendance, previous marks, sleep, tutoring, parental involvement and other factors affecting examination performance. The downloaded dataset contains 6,607 records and 20 columns, with Exam_Score as the final score.

Dataset source: [Student Performance Factors – Kaggle](#)

Algorithm / Pseudocode

1. Load the student-performance dataset.
2. Select attendance, study hours and other relevant features.
3. Clean missing, duplicate and invalid records.
4. Convert examination scores into pass and at-risk classes.
5. Split the records into training and testing datasets.
6. Train a Gaussian Naive Bayes classifier.
7. Evaluate the model and predict a new student's performance.

Python Code from

```
pathlib import Path
```

```
import pandas as pd from
sklearn.metrics import (
accuracy_score,
classification_report,
confusion_matrix
)
from sklearn.model_selection import train_test_split from
sklearn.naive_bayes import GaussianNB print("Student
Performance Prediction Program Started")

# Locate dataset folder = Path(__file__).resolve().parent
dataset_path = folder / "StudentPerformanceFactors.csv"

if not dataset_path.exists():
    print("Dataset not found:", dataset_path)
raise SystemExit

# Load dataset data =
pd.read_csv( dataset_path,
encoding_errors="ignore",
on_bad_lines="skip"
)

print("Dataset loaded successfully") print("Original
dataset size:", data.shape)

# Select input features features
= [
    "Hours_Studied",
```

```

    "Attendance",
    "Sleep_Hours",
    "Previous_Scores",
    "Tutoring_Sessions",
    "Physical_Activity"
]
target = "Exam_Score"

data = data[features + [target]].copy()

# Convert values to numbers for
column in features + [target]:
data[column] = pd.to_numeric(
data[column],
errors="coerce"
)

# Clean dataset data =
data.dropna() data =
data.drop_duplicates()

data = data[data["Attendance"].between(0, 100)] data
= data[data["Previous_Scores"].between(0, 100)] data
= data[data["Exam_Score"].between(0, 100)] data =
data[data["Sleep_Hours"].between(0, 24)]

# Create classification target
data["Performance"] = (
data["Exam_Score"] >= 60

```

```
).astype(int)

print("Dataset size after cleaning:", data.shape)

print("\nPerformance distribution:") print(data["Performance"].value_counts())

print("0 = At Risk") print("1
= Pass")

# Input and output X =
data[features] y =
data["Performance"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.20,
random_state=42, stratify=y
)

print("\nTraining records:", len(X_train)) print("Testing
records:", len(X_test))

# Train model model =
GaussianNB()
model.fit(X_train, y_train)

print("\nModel trained successfully")
```

```

# Evaluate model predictions =
model.predict(X_test) print(

    "Model Accuracy:",
    round(accuracy_score(y_test, predictions) * 100, 2),
    "%")
)

print("\nConfusion Matrix:") print(confusion_matrix(y_test,
predictions))

print("\nClassification Report:") print(
classification_report(    y_test,
predictions,    target_names=["At
Risk", "Pass"],    zero_division=0
    )
)

# Get new student details print("\nEnter
New Student Details")

try:
    hours_studied = float(
input("Hours studied per week: ")
    )
    attendance = float(
input("Attendance percentage: ")    )
    sleep_hours = float(    input("Average
sleep hours per day: ")

```

```

    )
    previous_scores = float(
input("Previous examination score: ")
    )
    tutoring_sessions = float(
input("Number of tutoring sessions: ")
    )
    physical_activity = float(
input("Physical activity hours per week: ")
    )

except ValueError:    print("Enter
numerical values only.")    raise
SystemExit

new_student = pd.DataFrame(
    [[
        hours_studied,
attendance,    sleep_hours,
previous_scores,
tutoring_sessions,
physical_activity
    ]],
    columns=features
)

prediction = model.predict(new_student)[0] probabilities
= model.predict_proba(new_student)[0]

print(

```

```
        "\nAt-Risk Probability:",
        round(probabilities[0] * 100, 2),
        "%"
    )

    print(
        "Pass Probability:",
        round(probabilities[1] * 100, 2),
        "%"
    )

    if prediction == 1:
        print("Predicted Performance: PASS") else:
        print("Predicted Performance: AT RISK")
```

Sample Input

Hours studied per week: 25

Attendance percentage: 92

Average sleep hours per day: 8

Previous examination score: 82

Number of tutoring sessions: 2

Physical activity hours per week: 3

Output Screenshot

```
----- MODEL EVALUATION -----
Model Accuracy: 99.09 %

Confusion Matrix:
[[  3  11]
 [  1 1306]]

Classification Report:
              precision    recall  f1-score   support

   At Risk      0.75      0.21      0.33         14
    Pass      0.99      1.00      1.00       1307

   accuracy              0.99       1321
  macro avg              0.87      0.61      0.66       1321
 weighted avg              0.99      0.99      0.99       1321


Sample Predictions:
Actual Result Predicted Result
0          Pass           Pass
1          Pass           Pass
2          Pass           Pass
3          Pass           Pass
4          Pass           Pass
5          Pass           Pass
6          Pass           Pass
7          Pass           Pass
8          Pass           Pass
9          Pass           Pass

----- ENTER NEW STUDENT DETAILS -----
Hours studied per week: 25
Attendance percentage: 92
Average sleep hours per day: 8
Previous examination score: 82
Number of tutoring sessions: 2
Physical activity hours per week: 3

----- PREDICTION RESULT -----
At-Risk Probability: 0.0 %
Pass Probability: 100.0 %
Predicted Student Performance: PASS
Performance Level: Good
```

Result

The student-performance learning system was developed successfully. The model obtained an overall accuracy of **99.09%**. However, because the dataset contained 6,537 passing records and only 68 at-risk records, the at-risk recall was only **21%**. Therefore, the overall accuracy is high, but the dataset imbalance limits the system's ability to identify weak students.

Q4) Model the Whether forecasting system with its outcome and challenges using random variables and probability theory.

Problem Description

The objective is to model a weather-forecasting system using random variables and conditional probability. The current weather is represented by the random variable X_t , and the weather in the next hour is represented by X_{t+1} .

The system calculates:

$$P(X_{t+1} = j \mid X_t = i)$$

The next weather condition is predicted using the outcome with the highest transition probability.

Dataset Description

The **Weather in Szeged 2006–2016** dataset was uploaded by Norbert Budincsevity. It contains historical hourly and daily weather observations for Szeged, Hungary, including timestamp, summary, temperature, humidity, wind speed, visibility and atmospheric pressure.

Dataset source: [Weather in Szeged 2006–2016 – Kaggle](#)

Algorithm / Pseudocode

1. Load historical weather observations from the CSV file.
2. Clean timestamps, weather summaries and numerical measurements.
3. Convert detailed weather summaries into general weather states.
4. Calculate the probability distribution of each weather state.
5. Create current-state and next-state transition pairs.
6. Calculate conditional transition probabilities between states.
7. Predict the next weather using the highest probability outcome.

Python Code from

```
pathlib import Path
```

```
import pandas as pd
```

```
print("Weather Forecasting Using Probability Theory")
```

```
# Locate dataset folder =
```

```
Path(__file__).resolve().parent dataset_path
```

```
= folder / "weatherHistory.csv" if not
```

```
dataset_path.exists(): print("Dataset not  
found:", dataset_path) raise SystemExit
```

```
# Load dataset data =
```

```
pd.read_csv( dataset_path,
```

```
encoding_errors="ignore",
```

```
on_bad_lines="skip"
```

```
)
```

```
print("Dataset loaded successfully") print("Original  
dataset size:", data.shape)
```

```
# Select required columns columns
```

```
= [
```

```
"Formatted Date",
```

```
"Summary",
```

```
"Temperature (C)",
```

```
"Humidity",
```

```
"Wind Speed (km/h)",
```

```
"Pressure (millibars)"
```

```
]
```

```
data = data[columns].copy()
```

```

# Convert date data["Formatted Date"] =
pd.to_datetime( data["Formatted
Date"], errors="coerce", utc=True
)

# Convert numerical columns numeric_columns
= [
    "Temperature (C)",
    "Humidity",
    "Wind Speed (km/h)",
    "Pressure (millibars)"
]

for column in numeric_columns:
data[column] = pd.to_numeric(
data[column], errors="coerce"
)

data = data.dropna(
subset=[
    "Formatted Date",
    "Summary",
    "Temperature (C)",
    "Humidity"
]
)

data = data.drop_duplicates(
subset=["Formatted Date"] )

```

```

data = data.sort_values(
    "Formatted Date"
).reset_index(drop=True)

print("Dataset size after cleaning:", data.shape)

# Convert summary into weather states
def classify_weather(summary):
    summary = str(summary).lower()

    if "rain" in summary or "drizzle" in summary:
        return "Rainy"
    if "snow" in summary or
"sleet" in summary:
        return "Snowy"
    if "fog" in summary
or "mist" in summary:
        return "Foggy"
    if "windy" in summary or
"breezy" in summary:
        return "Windy"
    if "cloud" in summary or
"overcast" in summary:
        return "Cloudy"
    if "clear" in summary
or "dry" in summary:
        return "Clear"

    return "Other"

data["Weather_State"] = data["Summary"].apply(    classify_weather
)

```

```

# Weather probability distribution
distribution = (
data["Weather_State"]
    .value_counts(normalize=True)
    .mul(100)
    .round(2)
)

print("\nWeather Probability Distribution:") print(distribution)

# Random-variable statistics print(
    "\nExpected Temperature:",
round(data["Temperature (C)"].mean(), 2),
    "°C"
)

print(
    "Temperature Variance:",
round(data["Temperature (C)"].var(), 2)
)

print(    "Expected Humidity:",
round(data["Humidity"].mean(), 2) )

# Create next state data["Next_State"] =
data["Weather_State"].shift(-1) data["Next_Time"] =
data["Formatted Date"].shift(-1)

```

```

data["Time_Gap"] = ( data["Next_Time"] -
data["Formatted Date"]
).dt.total_seconds() / 3600

# Keep hourly transitions transitions =
data[
data["Time_Gap"].between(0.5, 1.5)
].dropna(subset=["Next_State"])

# Transition probability table
transition_probabilities = pd.crosstab(
transitions["Weather_State"],
transitions["Next_State"],  normalize="index"
).mul(100)

print("\nTransition Probability Table (%):") print(transition_probabilities.round(2))

# Forecast available_states =
transition_probabilities.index.tolist()

print("\nAvailable states:")
print(available_states)

current_weather = input(
    "Enter the current weather state: "
).strip().title()

if current_weather not in available_states:

```

```
    print("Invalid weather state.")
    raise SystemExit

forecast = (
    transition_probabilities
    .loc[current_weather]
    .sort_values(ascending=False)
)

print("\nNext-Weather Probabilities:")

for state, probability in forecast.items():
    print(state, ":", round(probability, 2), "%")

predicted_weather = forecast.idxmax()

print("\nMost Likely Next Weather:", predicted_weather) print("Probability:",
round(forecast.max(), 2), "%")

rain_probability = forecast.get("Rainy", 0) / 100

print(
    "P(Rain in next hour):",
    round(rain_probability * 100, 2),
    "%"
)

print(
```

```
"Rain Random Variable Variance:",  
round(rain_probability * (1 - rain_probability), 4)  
)
```

Output Screenshot

Number of valid hourly transitions: 96427

----- TRANSITION COUNTS -----					
Next_State	Clear	Cloudy	Foggy	Rainy	Windy
Current_State					
Clear	7703	2936	241	0	27
Cloudy	2988	71723	1198	17	653
Foggy	182	1260	5737	1	7
Rainy	0	17	1	94	0
Windy	34	643	10	0	955

----- TRANSITION PROBABILITY (%) -----					
Next_State	Clear	Cloudy	Foggy	Rainy	Windy
Current_State					
Clear	70.62	26.92	2.21	0.00	0.25
Cloudy	3.90	93.66	1.56	0.02	0.85
Foggy	2.53	17.53	79.82	0.01	0.10
Rainy	0.00	15.18	0.89	83.93	0.00
Windy	2.07	39.16	0.61	0.00	58.16

Available current weather states:

- Clear
- Cloudy
- Foggy
- Rainy
- Windy

Enter the current weather state: cloudy

Forecast probabilities when current weather is Cloudy:

Cloudy: 93.66 %
Clear: 3.9 %
Foggy: 1.56 %
Windy: 0.85 %
Rainy: 0.02 %

----- FORECAST RESULT -----
Most likely next weather: Cloudy
Probability: 93.66 %

----- RAIN RANDOM VARIABLE -----
P(Rain in next hour): 0.02 %
Expected value $E[R]$: 0.0002
Variance $\text{Var}(R)$: 0.0002

Outcome: Cloudy is the most likely weather condition in the next hour.

Result

The weather-forecasting system was modelled successfully using random variables and conditional transition probabilities. The system calculated $P(X_{t+1} | X_t)$ and selected the next weather state having the greatest probability.

Q6) Apply conditional probability concepts to design a fraud detection model for suspicious banking transaction.

Problem Description

The objective is to detect suspicious banking transactions using conditional probability and Gaussian Naive Bayes. The system first calculates the prior probability of fraud and then calculates the probability of fraud when a transaction has a high amount.

Let:

F = Fraudulent transaction

H = High-amount transaction

Bayes' theorem is applied as:

$$P(F | H) = \frac{P(H | F)P(F)}{P(H | F)P(F) + P(H | G)P(G)}$$

Dataset Description

The **Credit Card Fraud Detection** dataset contains European credit-card transactions recorded over two days in September 2013. It contains 284,807 transactions, including 492 fraud cases. Features V1 to V28 are anonymized PCA components; Time and Amount remain in their original forms, and Class is 1 for fraud and 0 for genuine transactions.

Dataset source: [Credit Card Fraud Detection – Kaggle](#)

Algorithm / Pseudocode

1. Load and clean the credit-card transaction dataset.
2. Calculate the prior probabilities of fraud and genuine transactions.
3. Define transactions above the 95th percentile as high amount.
4. Calculate $P(High | Fraud)$ and $P(High | Genuine)$.
5. Apply Bayes' theorem to calculate $P(Fraud | High)$.
6. Train Gaussian Naive Bayes using transaction features.
7. Evaluate the model and display the most suspicious transactions. **Python**

Code from pathlib import Path

```
import pandas as pd from
sklearn.metrics import (
accuracy_score,
average_precision_score,
classification_report,
confusion_matrix,
f1_score, precision_score,
recall_score
)
from sklearn.model_selection import train_test_split from
sklearn.naive_bayes import GaussianNB

print("Bank Transaction Fraud Detection Started")

# Locate dataset folder =
Path(__file__).resolve().parent
dataset_path = folder / "creditcard.csv"

if not dataset_path.exists():
    print("Dataset not found:", dataset_path)
    raise SystemExit

# Load dataset data =
pd.read_csv(dataset_path)

print("Dataset loaded successfully")
print("Original dataset size:", data.shape)
```

```

# Required features features
= ["Time"]

for number in range(1, 29):
    features.append("V" + str(number))

features.append("Amount")

data = data[features + ["Class"]].copy()
data = data.dropna() data =
data.drop_duplicates()

data["Class"] = data["Class"].astype(int)
data = data[data["Class"].isin([0, 1])] data
= data[data["Amount"] >= 0]

print("Dataset size after cleaning:", data.shape)

# Prior probabilities total = len(data)
fraud_count = (data["Class"] == 1).sum()
genuine_count = (data["Class"] == 0).sum()

p_fraud = fraud_count / total p_genuine
= genuine_count / total

print("\nPrior Probabilities:")
print("P(Fraud):", round(p_fraud * 100, 4), "%") print("P(Genuine):",
round(p_genuine * 100, 4), "%")

```

```

# Define high-amount event high_amount_limit =
data["Amount"].quantile(0.95)

data["High_Amount"] = (
data["Amount"] >= high_amount_limit
).astype(int)

fraud_data = data[data["Class"] == 1] genuine_data
= data[data["Class"] == 0]

p_high_given_fraud = fraud_data["High_Amount"].mean() p_high_given_genuine
= genuine_data["High_Amount"].mean()

print("\nHigh-Amount Threshold:", round(high_amount_limit, 2))

print(
    "P(High Amount | Fraud):",
    round(p_high_given_fraud * 100, 4),
    "%"
)

print(
    "P(High Amount | Genuine):",
    round(p_high_given_genuine * 100, 4),
    "%"
)

# Bayes theorem p_high = (
p_high_given_fraud * p_fraud

```

```
    + p_high_given_genuine * p_genuine  
)
```

```
if p_high > 0:  
    p_fraud_given_high = (  
p_high_given_fraud * p_fraud  
    ) / p_high else:  
    p_fraud_given_high = 0
```

```
print(  
    "\nP(Fraud | High Amount):",  
round(p_fraud_given_high * 100, 4),  
    "%"  
)
```

```
# Prepare ML data  
X = data[features] y  
= data["Class"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
X, y,  
    test_size=0.20,  
random_state=42, stratify=y  
)
```

```
# Train model model =  
GaussianNB()  
model.fit(X_train, y_train)
```

```

print("\nModel trained successfully")

# Prediction fraud_probabilities =
model.predict_proba(X_test)[:, 1] predictions =
(fraud_probabilities >= 0.50).astype(int)

# Evaluation print(  "Accuracy:",
round(accuracy_score(y_test, predictions) * 100, 2),
    "%")
)

print(
    "Fraud Precision:",
round(precision_score(
    y_test,
predictions,
zero_division=0
) * 100, 2),
    "%")
)

print(  "Fraud
Recall:",
round(recall_score(
    y_test,
predictions,
zero_division=0
) * 100, 2),
    "%")

```

```
)
```

```
print( "Fraud F1
```

```
Score:",
```

```
round(f1_score(
```

```
    y_test,
```

```
    predictions,
```

```
    zero_division=0
```

```
    ) * 100, 2),
```

```
    "%"
```

```
)
```

```
print( "Average Precision:",
```

```
round(
```

```
average_precision_score(
```

```
    y_test,
```

```
    fraud_probabilities
```

```
    ),
```

```
    4
```

```
    )
```

```
)
```

```
print("\nConfusion Matrix:") print(confusion_matrix(y_test,  
predictions))
```

```
print("\nClassification Report:") print(
```

```
classification_report(    y_test,
```

```
predictions,
```

```

target_names=["Genuine", "Fraud"],
zero_division=0
)
)

# Display suspicious transactions results
= pd.DataFrame({
    "Amount": X_test["Amount"].values,
    "Actual_Class": y_test.values,
    "Fraud_Probability": fraud_probabilities * 100,
    "Predicted_Class": predictions
})

results["Actual_Result"] = results["Actual_Class"].map({
    0: "Genuine",
    1: "Fraud"
})

results["Predicted_Result"] = results["Predicted_Class"].map({
    0: "Genuine",
    1: "Suspicious"
})

results["Fraud_Probability"] = (
    results["Fraud_Probability"].round(2)
)

most_suspicious = results.sort_values(
    "Fraud_Probability",
    ascending=False
)

```

```
).head(10)
```

```
print("\nMost Suspicious Transactions:")
```

```
print(
```

```
most_suspicious[
```

```
    [
```

```
        "Amount",
```

```
        "Actual_Result",
```

```
        "Fraud_Probability",
```

```
        "Predicted_Result"
```

```
    ]
```

```
].to_string(index=False)
```

```
)
```

Output Screenshot

```
----- BAYES THEOREM RESULT -----
P(Fraud | High Amount): 0.296 %

Training transactions: 226980
Testing transactions: 56746

Training Gaussian Naive Bayes model...
Model trained successfully

----- MODEL EVALUATION -----
Accuracy: 99.27 %
Fraud Precision: 13.77 %
Fraud Recall: 64.21 %
Fraud F1 Score: 22.68 %

Confusion Matrix:
[[56269  382]
 [   34   61]]

Classification Report:
              precision    recall  f1-score   support

   Genuine         1.00        0.99         1.00       56651
     Fraud         0.14        0.64         0.23         95

   accuracy                   0.99       56746
  macro avg         0.57        0.82         0.61       56746
 weighted avg         1.00        0.99         1.00       56746

----- MOST SUSPICIOUS TRANSACTIONS -----
Amount Actual_Result Fraud_Probability Predicted_Result
   45.51          Fraud              100.0       Suspicious
 1000.00       Genuine              100.0       Suspicious
 6454.74       Genuine              100.0       Suspicious
   380.00       Genuine              100.0       Suspicious
     1.00       Genuine              100.0       Suspicious
     2.29       Genuine              100.0       Suspicious
     1.00          Fraud              100.0       Suspicious
    75.86          Fraud              100.0       Suspicious
   523.35       Genuine              100.0       Suspicious
   723.21          Fraud              100.0       Suspicious

Program completed successfully
```

Result

The banking transaction fraud-detection model was implemented successfully using conditional probability, Bayes' theorem and Gaussian Naive Bayes. The system calculated the probability of fraud and displayed transactions with the highest fraud probabilities.

Because fraud records represent only a very small fraction of the dataset, precision, recall, F1-score and average precision are more meaningful than accuracy alone.

Q7) Illustrate how bayes decision theory can be applied in medical diagnosis for disease prediction

Problem Description

The objective is to apply Bayes decision theory to predict whether a breast tumour is benign or malignant. Gaussian Naive Bayes calculates the posterior probabilities of both diagnoses.

Bayes decision theory also considers the cost of an incorrect decision. Missing a malignant tumour is more dangerous than incorrectly marking a benign tumour as malignant. Therefore:

- False-positive cost = 1
- False-negative cost = 5

The decision with the minimum expected risk is selected.

Dataset Description

The **Breast Cancer Wisconsin Diagnostic Dataset** contains 569 patient samples and 30 numerical features computed from digitized images of fine-needle aspirates of breast masses. It contains 357 benign cases and 212 malignant cases. The features describe characteristics such as radius, texture, perimeter, area, smoothness, concavity and symmetry.

Dataset source: [Breast Cancer Wisconsin Diagnostic Dataset – Kaggle](#)

Original source: [UCI Machine Learning Repository](#)

Algorithm / Pseudocode

1. Load and clean the medical diagnosis dataset.
2. Convert benign and malignant outcomes into numerical classes.
3. Calculate prior probabilities for both diagnosis classes.
4. Train Gaussian Naive Bayes using tumour measurements.
5. Calculate posterior probabilities for every patient.
6. Calculate the risk of benign and malignant decisions.
7. Select the diagnosis with minimum expected decision risk. **Python Code**

```
from pathlib import Path
```

```
import numpy as np import
```

```
pandas as pd from
```

```
sklearn.metrics import (
```

```
accuracy_score,  
classification_report,  
confusion_matrix,  
precision_score,  
recall_score  
)  
  
from sklearn.model_selection import train_test_split  
from  
sklearn.naive_bayes import GaussianNB  
  
  
print("Medical Diagnosis Using Bayes Decision Theory")  
  
  
# Locate dataset folder =  
Path(__file__).resolve().parent  
dataset_path = folder / "data.csv"  
  
  
if not dataset_path.exists():  
    print("Dataset not found:", dataset_path)  
    raise SystemExit  
  
  
# Load dataset data =  
pd.read_csv(  
    dataset_path,  
    encoding_errors="ignore"  
)  
  
  
print("Dataset loaded successfully") print("Original  
dataset size:", data.shape)
```

```
# Remove empty, unnamed and ID columns data
= data.dropna(axis=1, how="all")
```

```
unnamed_columns = [ column for
column in data.columns if
column.lower().startswith("unnamed")
]
```

```
data = data.drop(
columns=unnamed_columns,
errors="ignore"
)
```

```
data = data.drop(
columns=["id"],
errors="ignore"
)
```

```
# Convert diagnosis
data["diagnosis"] = (
data["diagnosis"] .astype(str)
    .str.strip()
    .str.upper()
    .map({
        "B": 0,
        "M": 1
    })
)
```

```

data = data.dropna() data =
data.drop_duplicates() data["diagnosis"] =
data["diagnosis"].astype(int)

feature_columns = [ column for
column in data.columns if column
!= "diagnosis"
]

for column in feature_columns:
data[column] = pd.to_numeric(
data[column], errors="coerce"
)

data = data.dropna()

print("Dataset size after cleaning:", data.shape)

# Prior probabilities total = len(data)
benign_count = (data["diagnosis"] == 0).sum()
malignant_count = (data["diagnosis"] == 1).sum()
print("\nP(Benign):", round( benign_count /
total * 100,
2
), "%")

print("P(Malignant):", round(
malignant_count / total * 100,
2

```

```
), "%")
```

```
# Prepare data X =
```

```
data[feature_columns] y =
```

```
data["diagnosis"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y,
```

```
test_size=0.20,
```

```
random_state=42, stratify=y
```

```
)
```

```
# Train model model =
```

```
GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
print("\nModel trained successfully")
```

```
# Posterior probabilities
```

```
posterior = model.predict_proba(X_test)
```

```
classes = list(model.classes_) benign_index
```

```
= classes.index(0) malignant_index =
```

```
classes.index(1)
```

```
p_benign = posterior[:, benign_index] p_malignant
```

```
= posterior[:, malignant_index]
```

```

# Standard decision
standard_predictions = (
    p_malignant >= 0.50
).astype(int)

# Decision costs false_positive_cost
= 1 false_negative_cost = 5

# Conditional risk risk_decide_malignant
= ( false_positive_cost * p_benign
)

risk_decide_benign = (
    false_negative_cost * p_malignant
)

# Minimum-risk decision bayes_predictions
= np.where( risk_decide_malignant <=
    risk_decide_benign,
    1,
    0
)

bayes_threshold = (
    false_positive_cost
    / (
        false_positive_cost
        + false_negative_cost
    )
)

```

```
)

print(
    "Bayes Malignant Threshold:",
    round(bayes_threshold * 100, 2),
    "%"
)
```

```
# Evaluation function def
evaluate(title, actual, predicted):
    matrix = confusion_matrix(
        actual,
        predicted,
        labels=[0, 1]
    )
```

```
    tn, fp, fn, tp = matrix.ravel()

    specificity = (
        tn / (tn + fp)
        if tn + fp > 0
        else 0
    )
```

```
    decision_cost = (
        fp
        * false_positive_cost
        + fn * false_negative_cost
    )
```

```
print("\n-----", title, "-----")

print("Accuracy:", round(
accuracy_score(actual, predicted) * 100,
2
),
"%")
)
```

```
print(
    "Malignant Precision:",
    round(
precision_score(
actual, predicted,
zero_division=0

) * 100,
2
),
"%")
)
```

```
print(
    "Sensitivity:",
    round(
recall_score(
actual,
predicted,
zero_division=0

) * 100,
2
),
```

```

        "%")
    )
    print("Specificity:",
round(specificity * 100, 2),
        "%")
    )

    print("Decision Cost:", decision_cost)
print("\nConfusion Matrix:")    print(matrix)

    print("\nClassification Report:")    print(
classification_report(        actual,
predicted,        target_names=["Benign",
"Malignant"],        zero_division=0
    )
    )

    return decision_cost


standard_cost = evaluate(
    "STANDARD 50% DECISION",
    y_test,
    standard_predictions
)

bayes_cost = evaluate(
    "BAYES MINIMUM-RISK DECISION",

```

```

        y_test,
        bayes_predictions
    )

    print("\nStandard Decision Cost:", standard_cost) print("Bayes
    Decision Cost:", bayes_cost)

    if bayes_cost < standard_cost:
        print("Bayes decision produced lower medical risk.")

# Display high-risk patients results
results = pd.DataFrame({
    "Actual": y_test.values,
    "Benign_Probability": p_benign * 100,
    "Malignant_Probability": p_malignant * 100,
    "Bayes_Decision": bayes_predictions
})

results["Actual"] = results["Actual"].map({
    0: "Benign",
    1: "Malignant"
})

results["Bayes_Decision"] = results["Bayes_Decision"].map({
    0: "Benign",
    1: "Malignant"
})

```

```
results["Benign_Probability"] = (  
    results["Benign_Probability"].round(2)  
)
```

```
results["Malignant_Probability"] = (  
    results["Malignant_Probability"].round(2)  
)
```

```
print("\nHigh-Risk Patient Predictions:")
```

```
print(    results.sort_values(  
    "Malignant_Probability",  
    ascending=False  
    ).head(10).to_string(index=False)  
)
```

```
print("\nEducational model only—not for clinical use.")
```

Output Screenshot

```
----- BAYES MINIMUM-RISK DECISION -----
Accuracy: 93.86 %
Malignant Precision: 97.3 %
Sensitivity: 85.71 %
Specificity: 98.61 %
Decision cost: 31

Confusion Matrix:
[[71  1]
 [ 6 36]]

Classification Report:
      precision    recall  f1-score   support

   Benign         0.92      0.99      0.95         72
  Malignant         0.97      0.86      0.91         42

   accuracy              0.94         114
  macro avg              0.95      0.92      0.93         114
 weighted avg              0.94      0.94      0.94         114

----- COST COMPARISON -----
Standard decision cost: 35
Bayes decision cost: 31
Bayes decision theory produced a lower medical decision cost.

----- HIGH-RISK PATIENT PREDICTIONS -----
Actual_Diagnosis  Benign_Probability  Malignant_Probability  Risk_Decide_Benign  Risk_Decide_Malignant  Bayes_Decision
Malignant          0.0              100.0              5.0              0.0      Malignant
Malignant          0.0              100.0              5.0              0.0      Malignant
Malignant          0.0              100.0              5.0              0.0      Malignant
Malignant          0.0              100.0              5.0              0.0      Malignant
Malignant          0.0              100.0              5.0              0.0      Malignant
Malignant          0.0              100.0              5.0              0.0      Malignant
Malignant          0.0              100.0              5.0              0.0      Malignant
Malignant          0.0              100.0              5.0              0.0      Malignant
Malignant          0.0              100.0              5.0              0.0      Malignant
Malignant          0.0              100.0              5.0              0.0      Malignant
Malignant          0.0              100.0              5.0              0.0      Malignant

Program completed successfully
Educational model only - not for clinical diagnosis.
```

Result

The Gaussian Naive Bayes model achieved **93.86% accuracy**. The standard 50% decision missed seven malignant cases and produced a decision cost of 35.

Bayes decision theory reduced the malignant threshold to **16.67%**, increased sensitivity from **83.33% to 85.71%**, reduced the number of missed malignant cases from seven to six and reduced the decision cost from **35 to 31**. Therefore, Bayes minimum-risk decision theory produced a safer decision for this dataset.